

An aggressive algorithm for multiple string matching

Liuling Dai

Beijing Laboratory of Intelligent Information Technology, School of Computer Science, Beijing Institute of Technology, Beijing 100081, PR China

ARTICLE INFO

Article history:

Received 18 July 2008

Received in revised form 23 January 2009

Accepted 26 January 2009

Available online 30 January 2009

Communicated by M. Yamashita

Keywords:

String matching

Multiple string matching

Wu–Manber algorithm

Design of algorithms

ABSTRACT

A new algorithm based on the Wu–Manber algorithm for multiple string matching is presented in this paper. The algorithm eliminates the functional overlap of the table *HASH* and *SHIFT*, and computes the shift distances in an aggressive manner. After each test, the algorithm examines the character next to the scan window to maximize the shift distance. This idea is consistent with that of the quick-search (QS) algorithm. Experimental results on four alphabets show that the new algorithm is more efficient than Wu–Manber and other recent algorithms, particularly on short pattern sets and large alphabet.

© 2009 Elsevier B.V. All rights reserved.

1. Introduction

As a basic problem in computer science, multiple string matching aims to find all occurrences of all patterns from a finite set X in a text $T = T[1 \dots n]$ by a single-round scanning. The text T and patterns are sequences of characters from an alphabet Σ . In general, a string matching task is accomplished by many tests. In each test, the pattern strings are aligned with a substring of the text, referred to as scan window, to find matches. In a multiple string matching task, the length of a scan window equals to that of the shortest pattern string(s) in X . After one test, the scan window is shifted forward to perform another test until the end of the text is reached.

In this paper a new algorithm for multiple string matching is proposed based on the framework of the Wu–Manber algorithm (WM, in short) [14]. For description convenience, we name it as QMM which means quick multiple matching. By incorporating the ideas of the quick-search (QS, in short) algorithm [13], our algorithm improves the Wu–Manber algorithm by maximizing the shift distances. Actually, after each test, we compute the shift distance in an aggressive manner by examining the character next to the scan window. To justify this claim, in our experiments,

we compare our algorithm with the Wu–Manber algorithm and some recent algorithms on four kinds of alphabets. The results show that our algorithm is faster than Wu–Manber and recent algorithms in most cases, especially on short pattern sets and large alphabet.

The rest of the paper is organized as follows. Section 2 is about related works. In Section 3, details of the algorithm are presented. We also compare our algorithm with the Wu–Manber algorithm in this section. Then we present experimental results as well as discussion in Section 4 before concluding this paper in Section 5.

2. Related works

The problem of single string matching is the basis of the multi-string matching problem. For decades, the renowned Boyer–Moore (BM, in short) algorithm [2] has been frequently used for single-string matching. In the BM algorithm the characters are compared from right to left starting with the rightmost character of the pattern. In a case of mismatch it uses two functions, i.e., last occurrence function and good suffix function, and shifts the scan window by the maximum number of characters computed by these functions. The worst case running time of the BM algorithm is $O(mn)$, where m is the length of the pattern string. Some improved algorithms based on the BM algo-

E-mail address: dailiu@bit.edu.cn.

rithm have been presented [3,5,9], among which the quick-search algorithm [13] is an excellent one. The QS algorithm improves the BM algorithm by examining the character next to the scan window after each test to enlarge the average shift distance. In practice, the QS algorithm is very simple and fast.

The problem of multiple string matching has been studied very extensively. Aho and Corasick [1] presented a linear-time algorithm for multiple string matching problem, based on an automata approach. Commentz and Walter [4] presented an algorithm for the multiple pattern matching problem that combines the Boyer-Moore technique with the Aho-Corasick algorithm. The Commentz-Walter algorithm is substantially faster than the Aho-Corasick algorithm in practice. The Wu-Manber algorithm [13] is another extension of the BM algorithm proposed by Sun Wu and Udi Manber for multiple string matching. The Wu-Manber algorithm makes use of block-character which consists of B (often 2 or 3) characters as a working unit to judge possible matches and compute shift distances. The Wu-Manber algorithm hashes block-characters into integers with a function $h(\cdot)$, and uses hash tables to solve collisions. Because of its efficiency, it has been incorporated in the *agrep* command. Gonzalo Navarro and Mathieu Raffinot [12] had studied several algorithms besides the Aho-Corasick algorithm and the Wu-Manber algorithm. They have also analyzed the impact of pattern-length and alphabet-size on performance.

Some variants of the Wu-Manber algorithm have been proposed. Yang et al. [16] presented a variant of the Wu-Manber algorithm based on the QS bad-character. They used the table *HEAD* to decide whether a block-character is the prefix of some patterns, and fulfilled the table *SHIFT* with larger values. But because the table *HEAD* plays a similar role as the table *PREFIX* which is originally used in the Wu-Manber algorithm, the improvement on performance is limited. Zhou et al. [17] tuned the Wu-Manber algorithm to solve large scale pattern set problem. They introduced *multi-phase hash* to cut down memory requirement and used *dynamic-cut heuristics* to select optimum m , the width of the scan window, to get high performance. Under the circumstances of large scale pattern set, their algorithm is efficient and consumes comparatively less memory. But, according to their experiments, when the number of patterns is less than 30,000, the Wu-Manber algorithm manifests better performance. Recently, Thierry Lecroq [9] proposed a family of fast single string matching algorithms based on the Wu-Manber algorithm.

3. Algorithm

3.1. General concerns

For description convenience, we firstly define two notions.

Definition 1. For a given pattern set X , the *pattern-set-length* of X is the length of the shortest pattern of X .

Definition 2. For a given string s , the *tail-block-character* (*TBC*) of s is the substring that consists of s 's last B characters. B is the length of a block-character.

There are two hash tables in the Wu-Manber algorithm, i.e. *SHIFT* and *HASH*. Assuming that the *pattern-set-length* is m , when generating *SHIFT* and *HASH*, the Wu-Manber algorithm considers the first (or the last) m characters of each pattern. The item *HASH*[h] stores the patterns whose hash value on *TBC* is h . These candidate patterns need to be checked character by character against the text. The item *SHIFT*[h] stores the number of characters that the scan window can safely shift after a test when the hash value on *TBC* of the scan window is h . The size of the table *SHIFT* and *HASH* is often set to 32,678 or 65,536 in practice. In addition, the Wu-Manber algorithm uses another table named *PREFIX*, which stores the prefixes of patterns of X , to differentiate patterns that have same hash value on tail-block-character but with different prefixes. The length of a prefix is normally set to 2.

During matching stage, the shift distance after each test is at least one character. In other words, the character immediately after the scan window will always be considered in the next test. Enlightened by this fact, we argue that this character can be very helpful to compute the shift distance.

Definition 3. For a given scan window, its *subsequent-block-character* (*SBC*) is the string that consists of the last $B - 1$ characters of the scan window and the character after it.

To utilize the character after the current scan window, after each test, we compute the shift distance based on the *SBC* of the scan window. This idea accords to the QS algorithm.

Based on the framework of the Wu-Manber algorithm, our algorithm uses three tables, i.e., *SHIFT*, *HASH* and *PREFIX*. The purpose and initialization of *HASH* and *PREFIX* are exactly the same as their counterparts in the Wu-Manber algorithm. An exception is that, *SHIFT* is constructed in an aggressive manner to get larger shift distances. Another exception is that the shift distances are only decided by subsequent-block-characters, which means that they are independent to matching results. In practice, the new algorithm uses the table *HASH* as the entrance of further character-by-character comparisons and looks up *SHIFT* to get shift distances after tests.

We assume that the pattern-set-length of X is m and there are N patterns in X . In the preprocessing stage and the matching stage, the patterns are aligned at the beginning of them. If m is 2 or the product of m and N is less than 500, we set the size of *HASH* and *SHIFT* as 32,678 and B as 2. The table *PREFIX* is not used under such circumstance. Given block-character BC of length 2, the hash function is defined as:

$$h(BC) = (BC[0] \ll 8 + BC[1]) \% 32678. \quad (1)$$

If m is larger than 5 or the product of m ($2 < m < 5$) and N is larger than 500, then we set the size of *HASH* and *SHIFT* as 65,536 and B as 3. The table *PREFIX* is used to

accelerate matching. Given block-character BC of length 3, the hash function is defined as:

$$h(BC) = (BC[0] \ll 16 + BC[1] \ll 8 + BC[2]) \% 65536. \quad (2)$$

The length of prefixes is set to 2. Given block-character BC of length 2, the hash function to compute its prefix hash value is:

$$h(BC) = BC[0] \ll 8 + BC[1]. \quad (3)$$

In the following two subsections, we continue to present the details of the preprocessing stage and the matching stage.

3.2. Preprocessing

In the preprocessing stage, the three tables are initialized so that they can be used in the matching stage. Note that the initialization of the table $HASH$ and $PREFIX$ is the same as those of the Wu–Manber algorithm, so we focus on the initialization of the table $SHIFT$, which is significantly different to the Wu–Manber algorithm.

We use the first m characters of each pattern to construct $SHIFT$. First of all, let us analyze the maximum possible shift distance after a certain test. When computing the shift distance, we check the SBC of the current scan window. According to the occurrence of SBC in elements of X , the maximum shift distance after this test can be decided by the heuristic rules as follows.

Case 1. SBC is a substring of some patterns. Assuming that the rightmost position (the position of the first character of a string is 1) of SBC in these patterns is q ($B \leq q \leq m$), then the maximum safe shift distance could be $m - q + 1$ characters.

Case 2. SBC does not occur in any element of X . But there are suffixes of SBC that match the prefixes of some elements of X . Assuming that the maximum length of these matching substrings is L ($0 < L < B$), then the safe shift distance could be $m - L + 1$.

Case 3. Neither case 1 nor case 2. In this instance, the safe shift distance could be $m + 1$.

Fig. 1 illustrates these three cases.

To initialize the table $SHIFT$ with the instructions above, we firstly set all the item of $SHIFT$ as $m + 1$. Then we modify them by using the patterns in X . Suppose that $p \in X$ is a pattern in X , the new algorithm modifies items of $SHIFT$ with p in two steps:

Step 1. For each p 's substring of length B , $BC = a_{j-B+1} \dots a_j$ ($B \leq j \leq m$), compute its hash value h , and set $SHIFT[h]$ as $\min\{m - j + 1, SHIFT[h]\}$.

Step 2. If $B = 2$, for any block-character $BC = c_1 p[1]$, $c_1 \in \Sigma$, set $SHIFT[h(BC)]$ as $\min\{m, SHIFT[h(BC)]\}$. If $B = 3$, for any block-character $BC = c_1 p[1]p[2]$, set $SHIFT[h(BC)]$ as $\min\{m - 1, SHIFT[h(BC)]\}$; for any block-character $BC = c_1 c_2 p[1]$, set $SHIFT[h(BC)]$ as $\min\{m, SHIFT[h(BC)]\}$, $c_1, c_2 \in \Sigma$.

3.3. Matching

The size of the scan window is set to m in the matching stage, and the table $HASH$ is used as the criteria of possible matches at each test. Actually, there could be possible matches only when the hash value of the TBC of the scan window is registered in $HASH$. The corresponding item of $HASH$ stores the candidate patterns which may be successfully matched with the text. Otherwise, the corresponding item of $HASH$ is *null*. No matter a test succeeds or not, the SBC of the scan window is hashed into an integer. This integer is used to look up $SHIFT$ to get shift distance. The main loop of the matching stage is as follows.

- (1) Compute the hash value ht of TBC of the scan window;
- (2) Check $HASH[ht]$, if $HASH[ht]$ is *null*, go to 5;
- (3) Compute the hash value hp of the prefix of the scan window;
- (4) Iterate each pattern p stored in $HASH[ht]$. Check whether $PREFIX[p]$ is equal to hp . If it is true, compare p with the text character-by-character to p 's actual length. If there is a successful match, report it;
- (5) Compute the hash value hs of SBC of the scan window. Shift the scan window by $SHIFT[hs]$ characters forward. Loop to 1 until reaching the end of text T .

In step (5), because there are $B - 1$ common characters between SBC and TBC , we can compute hs with the help of ht :

$$hs = (ht \ll 8 + SBC[B - 1]) \% 65536. \quad (4)$$

3.4. Technical analysis and comparison

We firstly would like to confirm that the new algorithm can find all the matches. The algorithm looks up the table $HASH$ to check all of the possible matches. The table $HASH$ of QMM is exactly the same as that of the Wu–Manber algorithm. Thus we can assure that no match will be missed in a test. In addition, from Fig. 1 we know that the table $SHIFT$ considers all the possibilities of SBC 's occurrences in pattern strings. That is to say, all the shift distances are

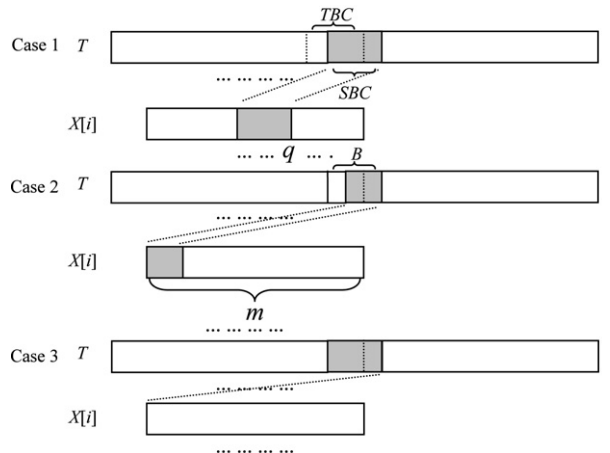


Fig. 1. The shift distance is decided by the occurrence of SBC in strings of X . There are 3 cases.

safe and no match will be missed when the scanning window is shifting.

We then analyze the computing complexity of the QMM algorithm. In the preprocessing stage, the computing complexity of Step 1 of the initialization of the table *SHIFT* is $O(M - B + 1)$. But the main time is consumed by Step 2. When $B = 2$, the computing complexity of the preprocessing stage is $O(N \bullet |\Sigma|)$. When $B = 3$, the computing complexity is $O(N \bullet |\Sigma|^2)$. In practice, the results of the preprocessing can be stored and reloaded, so, the computing complexity of matching stage is much more important. The best case time complexity of QMM is $O(n/m + 1)$. In this case there is no possible match. The worst case time complexity is $O(nM)$, where M is the length of the longest pattern of X . In this case, the shift distance after a test is always 1.

Although derived from the Wu–Manber algorithm, our algorithm is significantly different from it. We present the differences which make our algorithm more efficient as follows.

Firstly, in the Wu–Manber algorithm, there is a functional overlap between *HASH* and *SHIFT*. Actually, both the two tables can be used to judge whether there are possible matches. When an item of *SHIFT* is 0, the corresponding item of *HASH* is not null. On the contrary, when an item of *HASH* is null, the corresponding item of *SHIFT* must be larger than 0. This functional overlap results in low space efficiency and limits the shift distances. Differently, our algorithm strictly uses *HASH* to judge possible matches, and it uses *SHIFT* to get shift distances. There is no functional overlap between them, thus it is possible to enlarge shift distances during the matching stage.

Secondly, when the *TBC* of the scan window does not appear in any element of X , the Wu–Manber algorithm shifts the scan window by $m - B + 1$ characters. This shift distance is safe but conservative, which could make the Wu–Manber algorithm ineffective to certain degree. In our algorithm, under such a circumstance, the shift distance could be m or $m + 1$, larger than that of the Wu–Manber algorithm in practice.

Thirdly, the Wu–Manber algorithm concludes possible matches by observing whether certain items in *SHIFT* are 0. This strategy limits the shift distance to be only one character when there are possible or successful matches. If there are many possible or successful matches, the efficiency will be reduced notably. Alternatively, our algorithm concludes possible matches by observing whether certain items of the table *HASH* are null. While *SHIFT* is solely used to get shift distances, which are not related with matching results. In other words, the table *SHIFT* in our algorithm cannot be used to decide possible matches. This strategy makes shift distances only depend on the *SBCs* and range from 1 to $m + 1$ characters, which are larger than WM's in practice.

In brief, our algorithm eliminates the functional overlap between *SHIFT* and *HASH* in the Wu–Manber algorithm and maximizes the shift distances in an aggressive manner. These improvements result in fewer tests and thus less execution time.

4. Evaluation

4.1. Setup

In order to evaluate the efficiency of our algorithm, we carried out a series of experiments. Our algorithm was tested against four multiple string matching algorithms by running them with the same configurations. The algorithms are: the Wu–Manber algorithm (identified by WM), the Crochemore et al.'s algorithm (identified by MC) [6], the Xu et al.'s algorithm (identified by XB) [15], and the Yang et al.'s improved Wu–Manber algorithm (identified by YDH) [16]. The algorithms are tested on four kinds of texts:

- Alphabet of size 4. The text T is randomly generated from the DNA nucleotides: A, C, G and T. There are 4 mega characters in the text. In each test, the pattern set X is randomly generated.
- Alphabet of size 16. The text T is composed of 10 mega characters that are randomly selected from character A to P. The pattern sets are randomly generated.
- English text. We are intent to test the algorithms on a real-world English corpus. So, we used the Reuters corpus [10] as the text T of this configuration. There are 21,578 real-world news stories in this corpus with total size of 27,636,766 bytes. The alphabet consists of 79 different printable characters. In each test, the pattern set X was randomly constructed from a dictionary with 25,481 words.
- Chinese text. The text is a Chinese corpus in size of 52,277,012 bytes. The corpus is from the People's Daily newspaper of the first half year of 1998. The alphabet includes 162 different characters. All patterns to be matched were randomly selected from a Chinese dictionary with 79,570 words.

The Wu–Manber algorithm was extracted from Agrep [14]. Yang et al.'s algorithm and our algorithm were implemented based on the Wu–Manber algorithm. These three algorithms use the same hash techniques and the same hash table sizes. The Crochemore et al.'s algorithm and the Xu et al.'s algorithm were implemented with C++ language. For fair play, all the algorithms were tuned with the same techniques.

Tests were performed on a PC with Intel 2.1 GHz CPU and 1 GB memory. The operation system is Linux with kernel 2.6. To evaluate algorithms thoroughly, we run them with different pattern-set-length and different size of pattern set. Because we are intent to evaluate the performance of algorithms at usual circumstances, the top value of m is set as 12 and the top value of $|X|$ is set as 1000. In each run, the pattern set X is randomly generated and no repeated patterns were included. As the preprocessing stage can be performed offline and the result could be stored and loaded quickly, we are only interested in CPU time consumed by the matching stage of algorithms. For each configuration, algorithms were executed 15 times and the average scanning time was recorded.

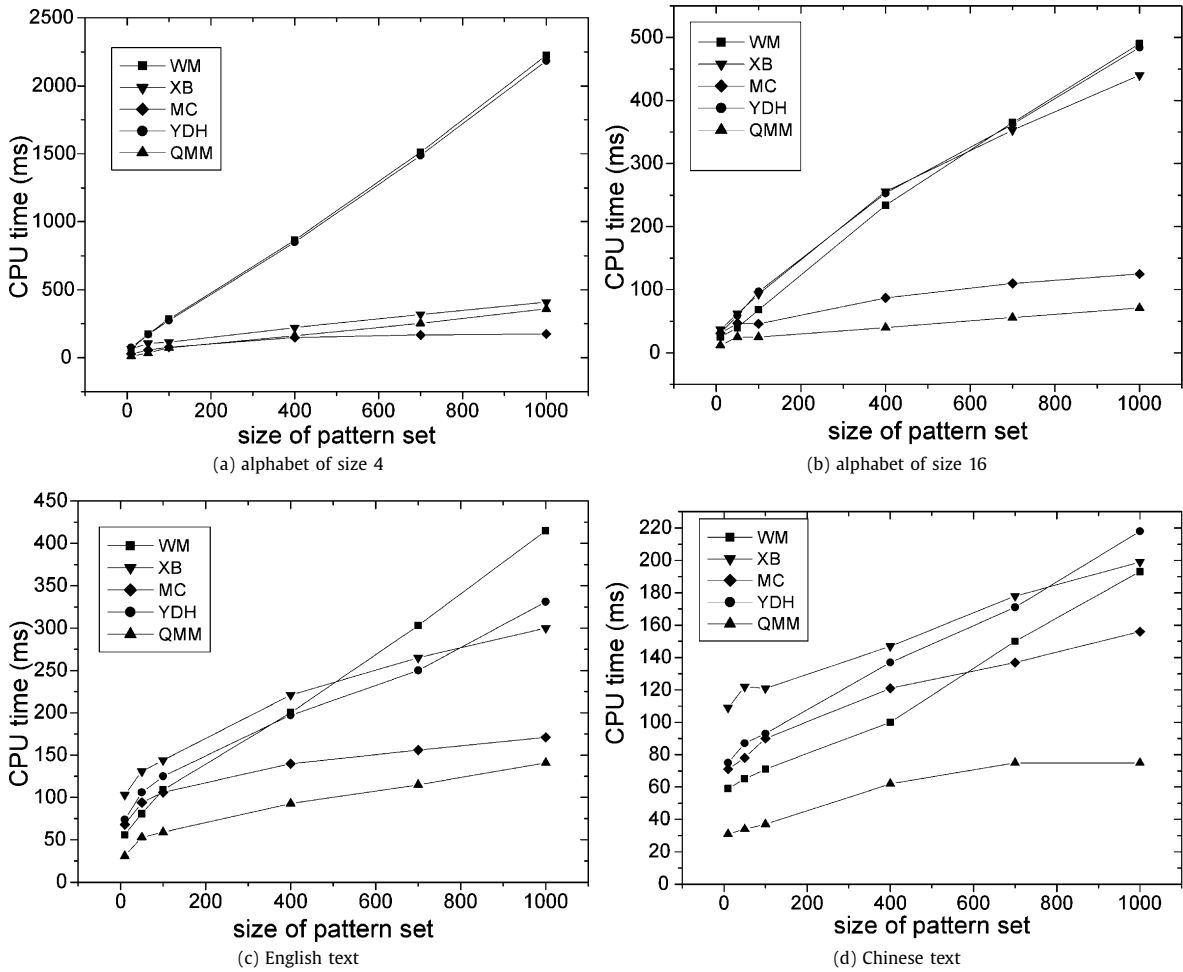


Fig. 2. Execution time against size of string set, on four kinds of text.

4.2. Experimental results

In this section we present the experimental results. Fig. 2 shows the scanning time of algorithms against the size of pattern sets. In Fig. 2, m is set as 6, while $|X|$ varies from 10 to 1000.

It can be seen from Fig. 2 that with the increase of the pattern-set size, the scanning time of algorithms increases. Fig. 2(a) shows that on text of alphabet of size 4, the Crochemore et al.'s algorithm and QMM are of top performance. The Crochemore et al.'s algorithm is slightly faster than QMM. Figs. 2(b), 2(c) and 2(d) show that QMM is the fastest algorithm among these algorithms. The predominance of QMM is notable in the English text and Chinese text.

Fig. 3 shows the execution time of algorithms against pattern-set-length. In this figure, the size of pattern-set is set as 400 ($|X| = 400$), while m varies from 2 to 12. Note that the Bo Xu's algorithm does not process the pattern-set when m is less than 4, thus results of such configurations are not applicable.

It can be seen from Fig. 3(a) that on alphabet of size 4, Crochemore et al.'s algorithm is the fastest one. This is due to the fact that its two processes decrease the num-

ber of symbol comparisons. When m is 2, the performance of QMM is similar to that of WM and YDH. This is because they need to perform about the same number of symbol comparisons and the shift distance is one character in most tests. With the increase of m , the scanning time of QMM evidently decreases due to the increase of shift distances. When m is larger than 6, the performance of QMM is very high, only slightly inferior to that of the Crochemore et al.'s algorithm.

In Figs. 3(b), 3(c) and 3(d), the QMM is the fastest one. Especially when m is small, the predominance of QMM is very remarkable. With the increase of m , all algorithms become faster. This is because with larger m , they could get larger shift distances.

It can be seen from the above figures that the performance of all algorithms varies according to different alphabet. To clarify this relationship, we illustrate the scanning speed of algorithms against the alphabet size in Fig. 4. In Fig. 4, m is set as 6 and $|X|$ is 400.

From Fig. 4 we can conclude that with the increase of alphabet size, the performance of each algorithm increase correspondingly. Particularly, when $|\Sigma|$ is 2, the performance of all the algorithms is very low and the dif-

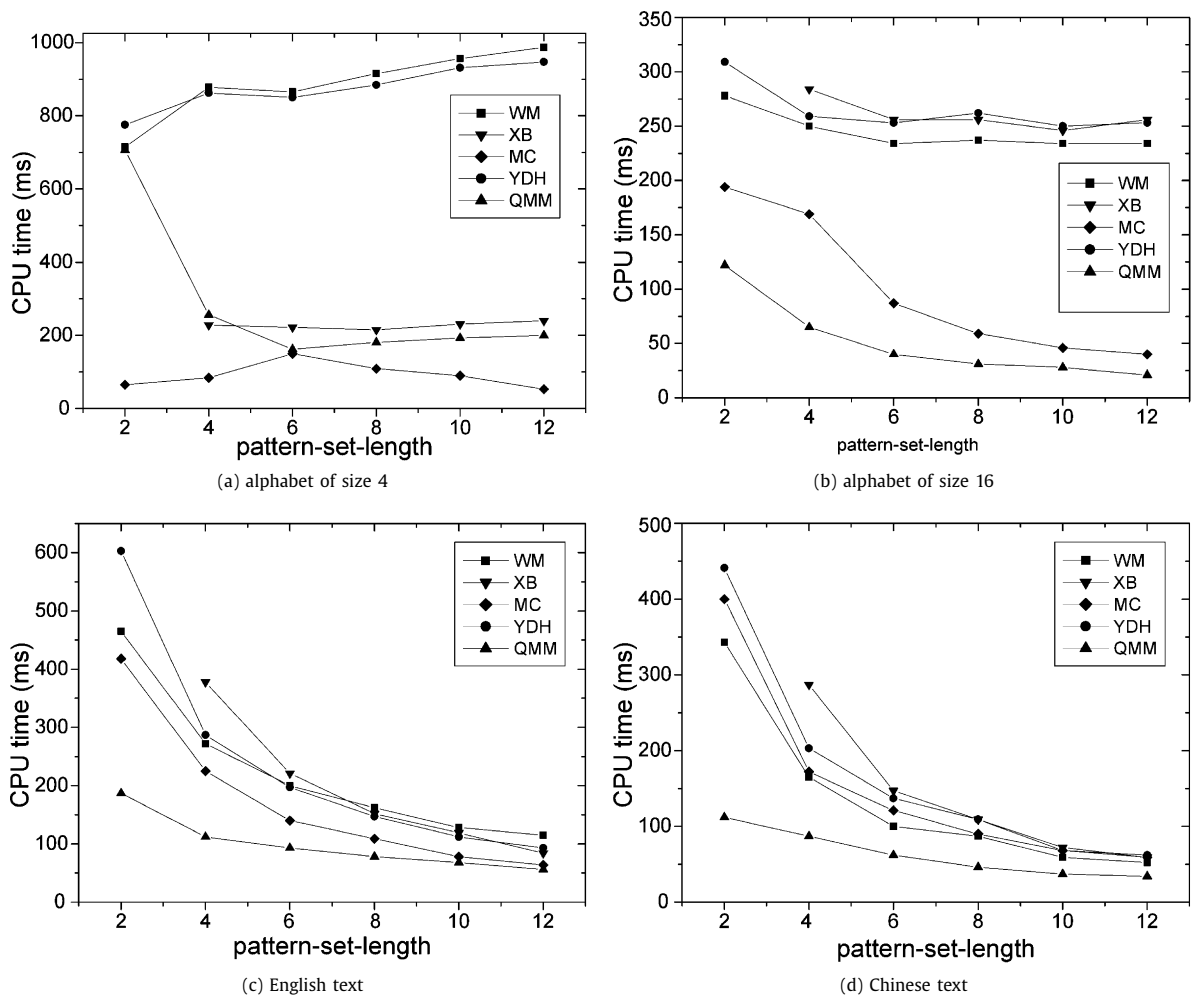


Fig. 3. Execution time of algorithms against pattern-set-length, on four kinds of text.

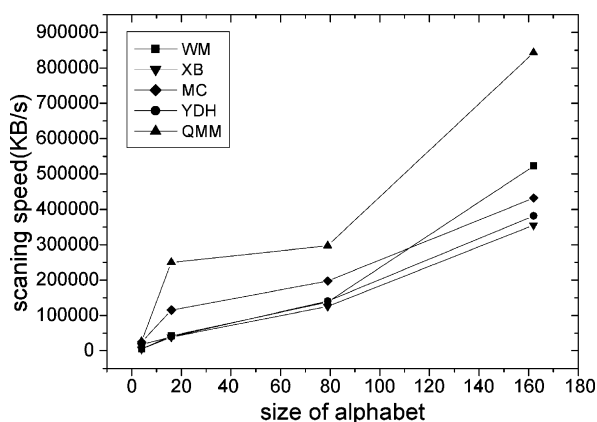


Fig. 4. Scanning speed against alphabet size.

ference among them is small. This is because with small alphabet and large pattern number, there are many possible matches during the matching stage. Algorithms need to perform many symbol comparisons, which yields low scanning speed. In Fig. 4, it is very clear that QMM is the

algorithm with the fastest scanning speed. In the case of alphabets whose size is larger than 4, the predominant of our algorithm is very notable.

4.3. Discussions

As shown in Figs. 2 and 3, scanning time of algorithms increases when the size of pattern set increases. In contrast, when the pattern-set-length increases, scanning time decreases. And algorithms run faster on larger alphabet. We conclude that under the circumstances of shorter pattern-set-length, smaller pattern-set size and larger alphabet, our algorithm will be more predominant when compared with the other 4 algorithms. For instance, in Fig. 3(c), where $|X|$ is 400 and the size of alphabet is 79, when m is 8, the CPU time of QMM is about 48% of WM's and 71% of the MC's. When m is 2, the CPU time of QMM is about 40% of WM's and 44% of MC's. In Fig. 4(c), where $|X|$ is 400 and the size of alphabet is 162, when m is 2, the CPU time of QMM is about 32% of WM's and 28% of the MC's.

From the experimental results we know that the SBC accelerates the algorithm by increasing the average shift

distance. On the other hand, in each test, the algorithm checks both the tables *HASH* and *SHIFT* to find possible matches and get shift distance. But if there are possible matches, the Wu–Manber algorithm does not look up the table *HASH* and directly move the window with 1 character, which avoid one memory accessing operation. So, there is a tradeoff between the increase of shift value and the increase of hash table checking. If there are many possible matches, the efficiency of QMM will degenerate, even inferior to WM. This situation subjects to little pattern-set-length, small alphabet and mass pattern strings. Also, this is exemplified by Figs. 2(a) and 3(a). In Fig. 2(a), with alphabet of size 4 and pattern-set-length of 6, QMM behaves closed to WM. And with the increase of the size of pattern set, the difference decreases. While in Fig. 3(a), with alphabet of size 4, pattern-set-length of 2 and pattern-set of size 400, the execution time of QMM and WM are very similar.

In addition, we compare our algorithm with the Kim et al.'s algorithm [8] and the Liu et al.'s algorithm [11] according to the results published in their papers. Kim et al.'s experiments on English text show that their algorithm is faster than the Wu–Manber algorithm. When $|X|$ is 10 and m is 3, CPU time of their algorithm (MULTI2) is about 80% of WM's. And when $|X|$ is 500 and m is 3, the proportion is about 61% (MULTI1). With the same configuration, the CPU time of our algorithm is about 34% and 52% of WM's. Although the running time may be influenced by the computer architecture [7], we can deduce that our algorithm is faster than Kim et al.'s algorithm. Liu et al. has tested their algorithm on the SNORT dataset and the CLAMAV dataset. The length of most patterns in the SNORT dataset ranged from 2 to 8 [11]. That is to say, the distribution of pattern length in this dataset is much similar to the real-world Reuters dataset used in our experiments. As shown in [11], the matching speed of the Liu et al.'s algorithm on SNORT is almost the same as WM's. So our algorithm is much faster than the Liu et al.'s algorithm on real-world datasets such as the Reuters and the SNORT.

5. Conclusion

In this paper we presented an improved algorithm for multiple string matching based on the framework of the Wu–Manber algorithm. The new algorithm eliminates the functional overlaps between the tables *HASH* and *SHIFT* in the Wu–Manber algorithm. And it constructs the table *SHIFT* in an aggressive manner by combining the idea of the QS algorithm. To maximize the shift distance after each test, the algorithm makes full use of the character next to the scan window. Extensive experiments proved the efficiency of the new algorithm. The new algorithm can be used in various applications, such as information filter and instruction detection.

Acknowledgements

This work is partially supported by Beijing Key Discipline Program, National Natural Science Foundation of China (60803050), and Basic Research Foundation of Beijing Institute of Technology (20071142004).

References

- [1] A.V. Aho, M.J. Corasick, Efficient string matching: an aid to bibliographic search, *Communications of the ACM* 18 (6) (1975) 333–340.
- [2] R.S. Boyer, J.S. Moore, A fast string searching algorithm, *Communications of the ACM* 20 (10) (1977) 762–772.
- [3] D. Cantone, S. Faro, Fast-search: A new efficient variant of the Boyer–Moore string matching algorithm, in: K. Jansen, M. Margraf, M. Mastrolilli, J.D.P. Rolim (Eds.), *Proceedings of the 2nd International Workshop on Experimental and Efficient Algorithms*, Ascona, Switzerland, 2003, in: *Lecture Notes in Computer Science*, vol. 2647, Springer-Verlag, Berlin, 2003, pp. 47–58.
- [4] B. Commentz-Walter, A string matching algorithm fast on the average, in: *Proceedings of the 6th Colloquium, on Automata, Languages and Programming*, July 16–20, 1979, pp. 118–132.
- [5] C. Charras, T. Lecroq, *Handbook of Exact String Matching Algorithms*, King's College London Publications, 2004.
- [6] M. Crochemore, A. Czumaj, L. Gasieniec, T. Lecroq, W. Plandowski, W. Rytter, Fast practical multi-pattern matching, *Information Processing Letters* 71 (3) (1999) 107–113.
- [7] P. Fenwick, Some perils of performance prediction: A case study on pattern matching, *Software Practice and Experience* 31 (9) (2001) 835–843.
- [8] S. Kim, Y. Kim, A fast multiple string-pattern matching algorithm, in: *Proc. 17th AoM/IAoM Conference on Computer Science*, San Diego, CA, August, 1999.
- [9] T. Lecroq, Fast exact string matching algorithms, *Information Processing Letters* 102 (6) (2007) 229–235.
- [10] D.D. Lewis, <http://www.daviddlewis.com/resources/testcollections/reuters21578/reuters21578.tar.gz>, 2004.
- [11] P. Liu, Y.B. Liu, J.L. Tan, A partition-based efficient algorithm for large scale multiple-strings matching, in: *Proc. of SPIRE'05*, in: LNCS, vol. 3772, 2005.
- [12] G. Navarro, M. Raffinot, *Flexible Pattern Matching in Strings: Practical On-Line Search Algorithms for Texts and Biological Sequences*, 1st edition, Cambridge University Press, 2007, July 30.
- [13] D.M. Sunday, A very fast substring search algorithm, *Communications of the ACM* 33 (8) (1990) 132–142.
- [14] S. Wu, U. Manber, A fast algorithm for multi-pattern searching, Technical Report TR-94-17, Department of Computer Science, University of Arizona, Tucson, AZ, 1994.
- [15] B. Xu, X. Zhou, J. Li, Recursive shift indexing: A fast multi-pattern string matching algorithm, in: *Proc. of the 4th International Conference on Applied Cryptography and Network Security (ACNS)*, 2006.
- [16] D.H. Yang, K. Xu, Y. Cui, An improved Wu–Manber multiple patterns matching algorithm, in: *Proc. of the 25th IEEE International Performance Computing and Communications Conference, IPCCC 2006*, Phoenix, Arizona, USA, April 10–12, 2006.
- [17] Z. Zhou, Y. Xue, J. Liu, W. Zhang, J. Li, MDH: A high speed multi-phase dynamic hash string matching algorithm for large-scale pattern set, in: *Proc. of 9th International Conference on Information and Communications Security (ICICS)*, in: LNCS, vol. 4861, 2007, pp. 201–215.